

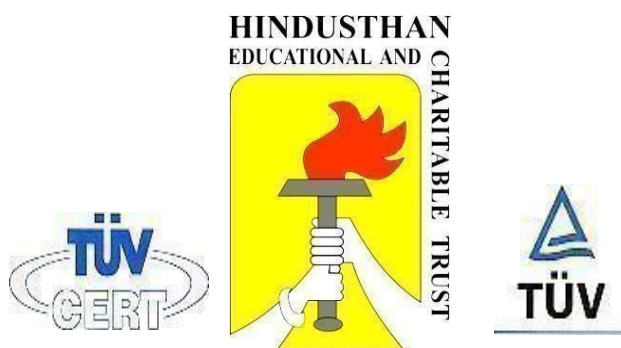
HINDUSTHAN COLLEGE OF ARTS & SCIENCE (AUTONOMOUS)

An Autonomous Institution - Affiliated to Bharathiar University

(ISO 9001-2001 Certified Institution)

Behind Nava India, Coimbatore – 641028.

DEPARTMENT OF COMPUTER APPLICATIONS (PG)



MASTER OF COMPUTER APPLICATIONS

PRACTICAL RECORD

25MCP20TP – PRACTICAL: AUGMENTED REALITY AND VIRTUAL REALITY (T & P)

NAME : _____

REGISTER NO : _____

CLASS : _____

SEMESTER : _____

YEAR : _____

HINDUSTHAN COLLEGE OF ARTS & SCIENCE (AUTONOMOUS)

An Autonomous Institution - Affiliated to Bharathiar University

(ISO 9001-2001 Certified Institution)

Behind Nava India, Coimbatore – 641028.

DEPARTMENT OF COMPUTER APPLICATIONS (PG)

25MCP20TP – PRACTICAL: AUGMENTED REALITY AND VIRTUAL REALITY (T & P)

CERTIFICATE

Certified that this is a bonafide record of **AUGMENTED REALITY AND VIRTUAL REALITY (T & P) (25MCP20TP)** done by_____

Register No:_____during the academic year 2026-2027.

STAFF IN - CHARGE

HEAD OF THE DEPARTMENT

Submitted for the Bharathiar University Practical Examination held on _____
at Hindusthan College of Arts & Science, Coimbatore - 28.

INTERNAL EXAMINER

EXTERNAL EXAMINER

Date:

Place: Coimbatore

CONTENTS

S.NO	DATE	NAME OF THE PROGRAM	PAGE NO	SIGN
01		INSTALL UNITY, UNITY BASICS, GAME OBJECTS, COMPONENTS, PREFABS		
02		CREATE INTERACTIVE SCENE, PHYSICS, UI BASICS		
03		SETUP XR PLUGIN (ANDROID), CROSS-PLATFORM SCENE		
04		AR CAMERA SETUP, PLACE OBJECTS ON PLANES		
05		IMAGE TRACKING DEMO, AR UI INTEGRATION		
06		VR SETUP (OCULUS/PC), ADD VR RIG, TELEPORTATION		
07		BUILD VR ENVIRONMENT, ADD INTERACTIVE OBJECTS AND UI		
08		ADD SPATIAL SOUND, OPTIMIZE MOBILE PERFORMANCE		
09		UI MENU IN AR/VR, INPUT MODULES, AR DEMO USING ALTERNATE WORKFLOWS BUILD AR DEMO (POSTER, CARD, MUSEUM, ETC...), CREATE VR SCENE (GALLERY, ESCAPE ROOM, ETC...)		
10		PRESENTATION, DEMO DAY, FINAL TOUCHES		

EX.NO: 01 DATE:	INSTALL UNITY, UNITY BASICS, GAME OBJECTS, COMPONENTS, PREFABS	PAGE NO:
----------------------------------	---	-----------------

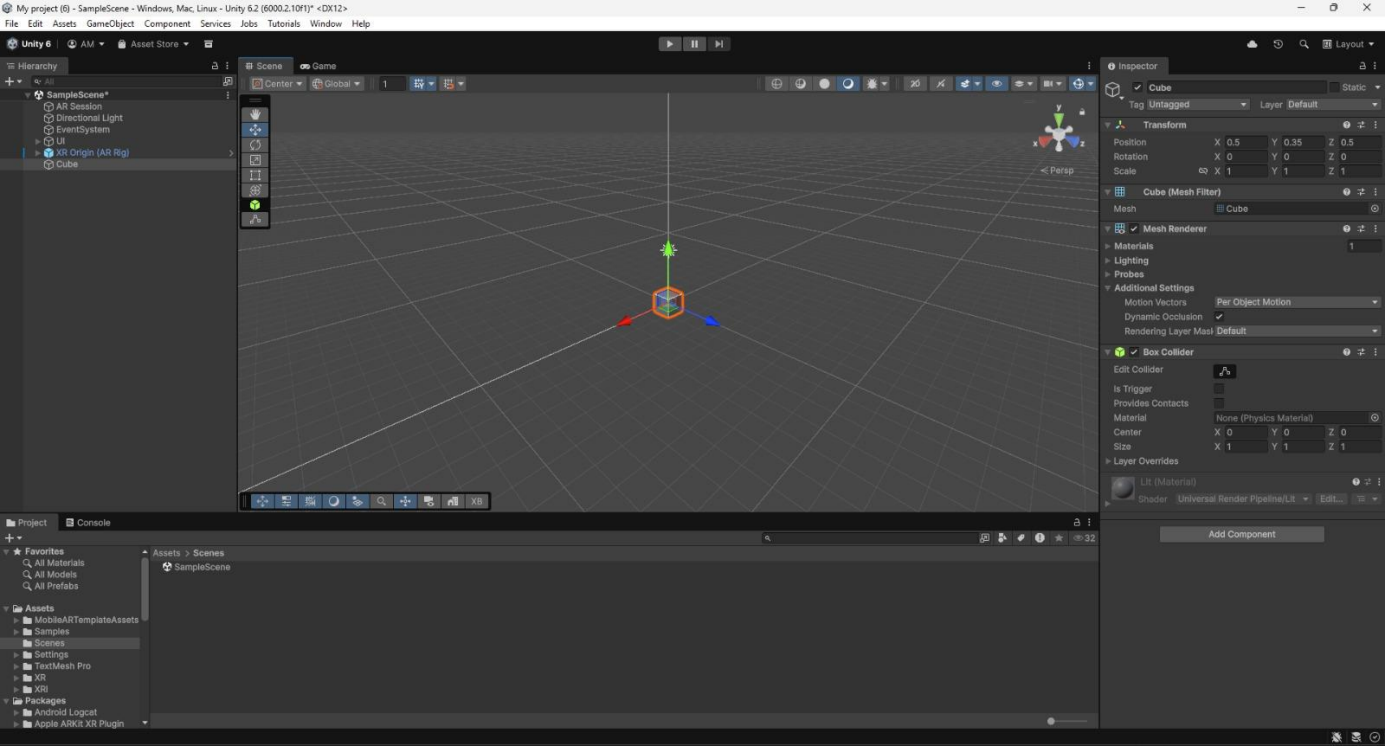
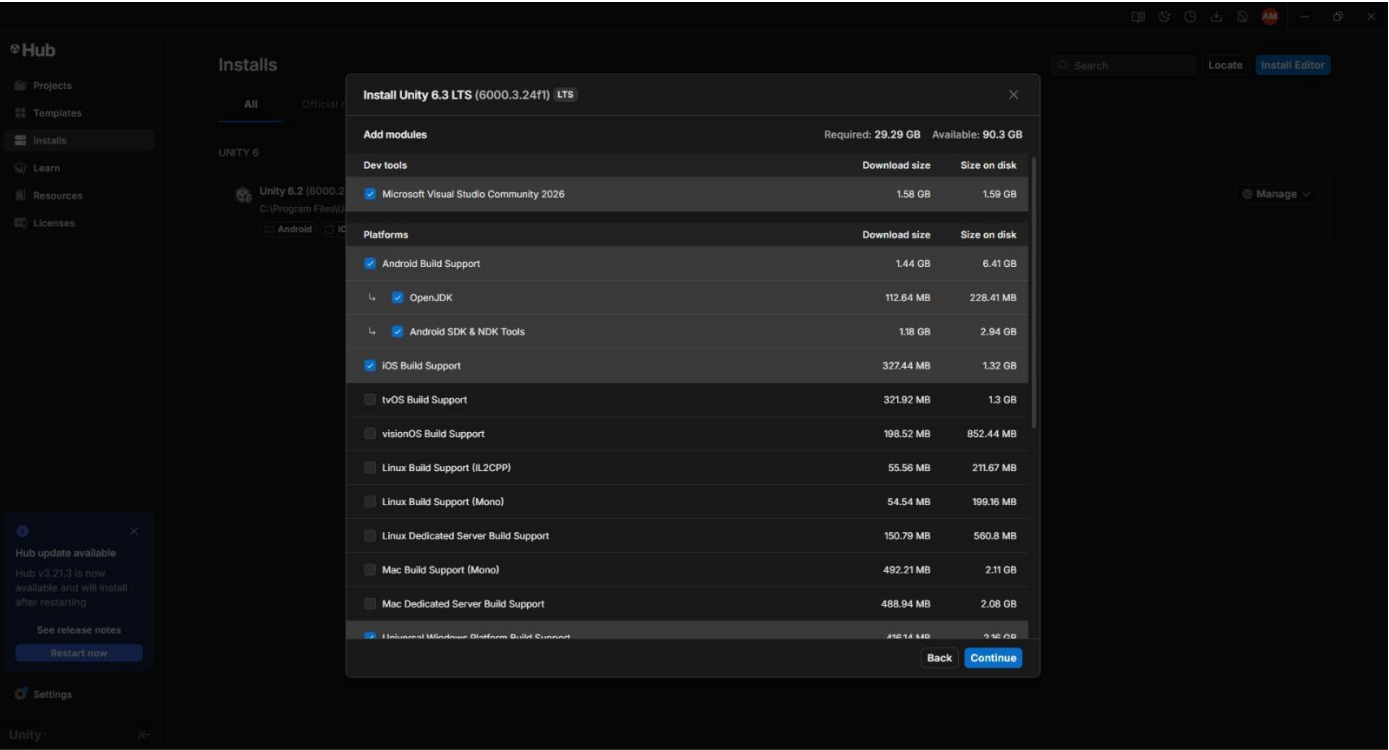
AIM:

To install Unity and study the basics of Unity, including Game Objects, Components, and Prefabs.

PROCEDURE:

1. Start the computer and ensure that the required system specifications are available for Unity.
2. Download and install Unity Hub from the official Unity website.
3. Open Unity Hub and sign in or create a Unity account.
4. Install the required Unity Editor version through Unity Hub.
5. Create a new 3D project using the installed Unity Editor.
6. Open the Unity project and observe the main Unity Editor interface.
7. Study the Scene View, Game View, Hierarchy, Inspector, and Project windows.
8. Create a Game Object such as a Cube, Sphere, or Plane in the scene.
9. Select the Game Object and observe its properties in the Inspector window.
10. Study the Components attached to the Game Object, such as Transform, Mesh Renderer, and Collider.
11. Add or modify Components using the Inspector window and observe their effect on the Game Object.
12. Create a Prefab by dragging a Game Object from the Hierarchy into the Project window.
13. Drag the created Prefab into the Scene to create multiple instances and observe their behavior.
14. Save the project and scene, then run the project to verify the created Game Objects, Components, and Prefabs.

OUTPUT:



RESULT:

Thus, Unity was successfully installed and the basics of Unity, including Game Objects, Components, and Prefabs, were studied and demonstrated successfully.

EX.NO: 02 DATE:	CREATE INTERACTIVE SCENE, PHYSICS, UI BASICS	PAGE NO:
----------------------------------	---	-----------------

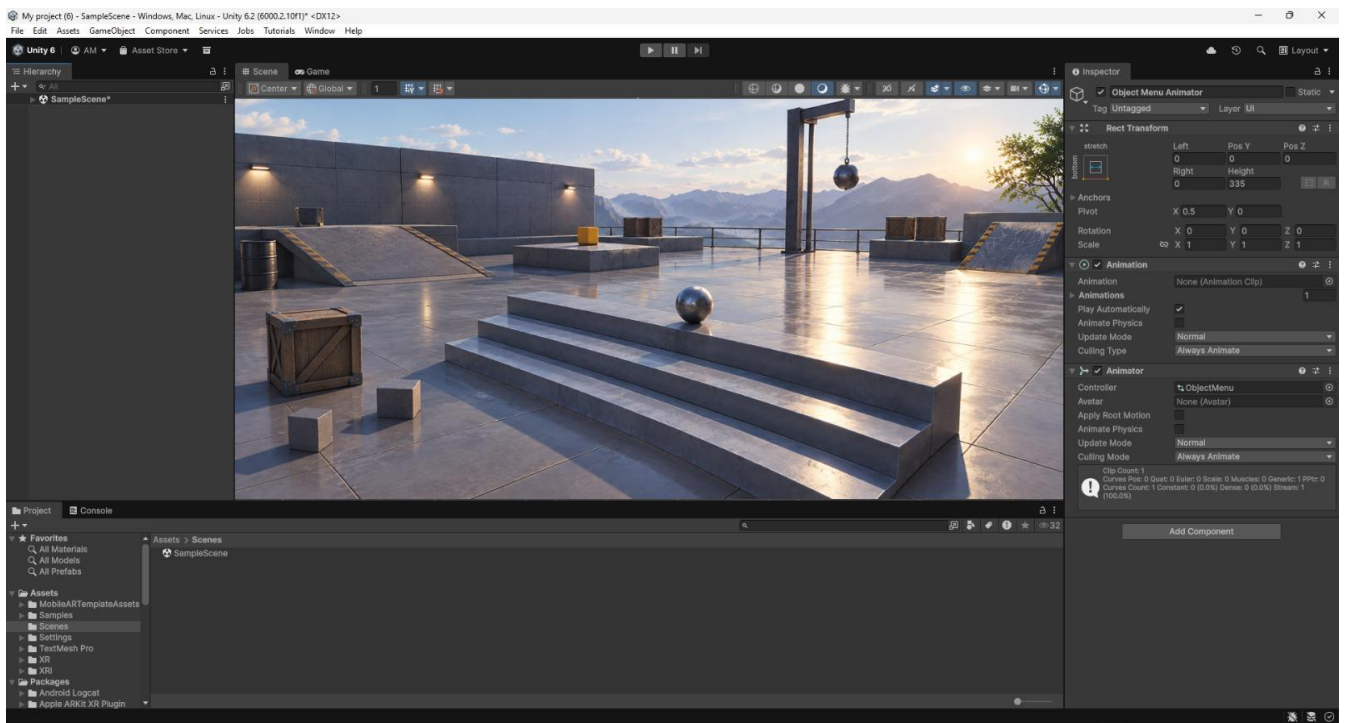
AIM:

To create an interactive scene in Unity by using Physics and basic UI elements such as buttons, text, and panels.

PROCEDURE:

1. Start Unity Hub and open the previously created Unity project.
2. Create a new scene and save it with a suitable name.
3. Add Game Objects such as a Plane, Cube, Sphere, and other required objects to the scene.
4. Arrange the Game Objects in the Scene View to create an interactive environment.
5. Add a Rigidbody component to the required Game Objects to enable physics-based movement.
6. Add Collider components such as Box Collider or Sphere Collider to the objects.
7. Adjust the Rigidbody properties such as Mass, Drag, and Gravity according to the scene requirements.
8. Run the scene and observe the physical behavior of the Game Objects.
9. Create a Canvas to provide a user interface for the scene.
10. Add UI elements such as Text, Button, Image, and Panel to the Canvas.
11. Arrange and customize the UI elements using the Inspector window.
12. Configure the Button to perform an action when it is clicked.
13. Test the interactive scene by interacting with the Game Objects and UI elements during Play mode.
14. Save the scene and project, then verify that the Physics and UI features work correctly.

OUTPUT:



RESULT:

Thus, an interactive Unity scene was successfully created using Physics and basic UI elements. The Game Objects responded to physics effects, and the UI elements such as Text, Buttons, and Panels were successfully added and tested.

EX.NO: 03	SETUP XR PLUGIN (ANDROID), CROSS- PLATFORM SCENE	PAGE NO:
DATE:		

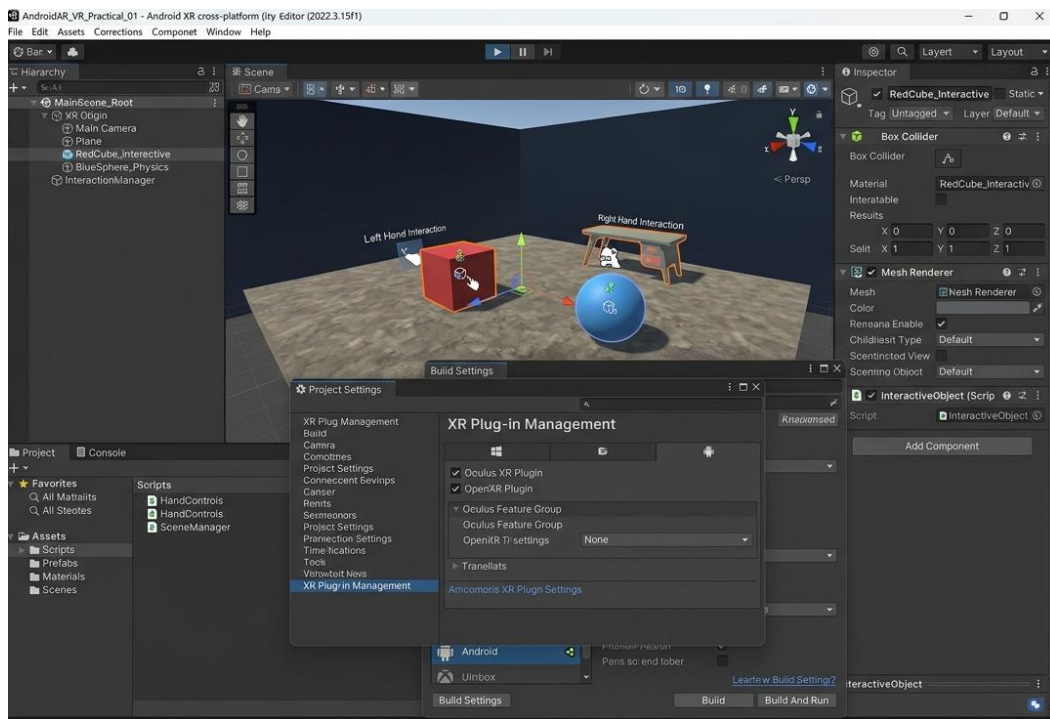
AIM:

To set up the XR Plugin for Android in Unity and create a cross-platform scene that supports XR applications on different platforms.

PROCEDURE:

1. Start Unity Hub and open the required Unity project.
2. Open the Package Manager from the Unity Editor.
3. Install XR Plug-in Management from the Unity Registry.
4. Open Project Settings and select XR Plug-in Management.
5. Enable the Android platform and configure the required XR settings.
6. Install the appropriate XR Plug-in provider required for the Android XR device.
7. Switch the project platform to Android using Build Settings.
8. Configure Android Player Settings such as package name, orientation, and minimum API level.
9. Create a new scene or open an existing scene for cross-platform development.
10. Add required XR components such as an XR Origin and camera setup to the scene.
11. Add basic Game Objects such as a Plane, Cube, and other interactive objects.
12. Configure the scene so that the objects can be viewed and interacted with across supported platforms.
13. Save the scene and check the XR configuration for errors or missing dependencies.
14. Build and test the application for Android and verify that the cross-platform scene loads and functions correctly.

OUTPUT:



RESULT:

Thus, the XR Plug-in Management for Android was successfully configured in Unity, and a cross-platform XR scene was created and tested successfully.

EX.NO: 04	AR CAMERA SETUP, PLACE OBJECTS ON	PAGE NO:
DATE:	PLANES	

AIM:

To set up an AR Camera in Unity and place virtual objects on detected real-world planes using AR Foundation.

PROCEDURE:

1. Start Unity Hub and open the required AR project.
2. Open the Package Manager and install the required AR Foundation package.
3. Install the appropriate platform-specific XR plug-in for the target device.
4. Create or open an AR scene in the Unity Editor.
5. Add an AR Session to manage the AR experience.
6. Add an AR Session Origin/XR Origin to the scene.
7. Configure the AR Camera and add the required AR camera components.
8. Add an AR Plane Manager to detect horizontal and vertical surfaces.
9. Add an AR Raycast Manager to detect positions on the detected planes.
10. Create a virtual 3D object such as a cube, sphere, or furniture model.
11. Create a placement system to select a position on a detected plane using a screen tap.
12. Instantiate the virtual object at the detected plane position.
13. Run the AR application on a supported Android device and allow camera access.
14. Point the camera at a real-world surface, detect the plane, and place the virtual object on it.

PROGRAM:

```
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.XR.ARFoundation;
using UnityEngine.XR.ARSubsystems;

public class PlaceObjectOnPlane : MonoBehaviour
{
    public GameObject objectToPlace;

    private ARRaycastManager raycastManager;
    private static List<ARRaycastHit> hits = new List<ARRaycastHit>();

    void Awake()
    {
        raycastManager = GetComponent<ARRaycastManager>();
    }

    void Update()
    {
        if (Input.touchCount == 0)
            return;

        Touch touch = Input.GetTouch(0);

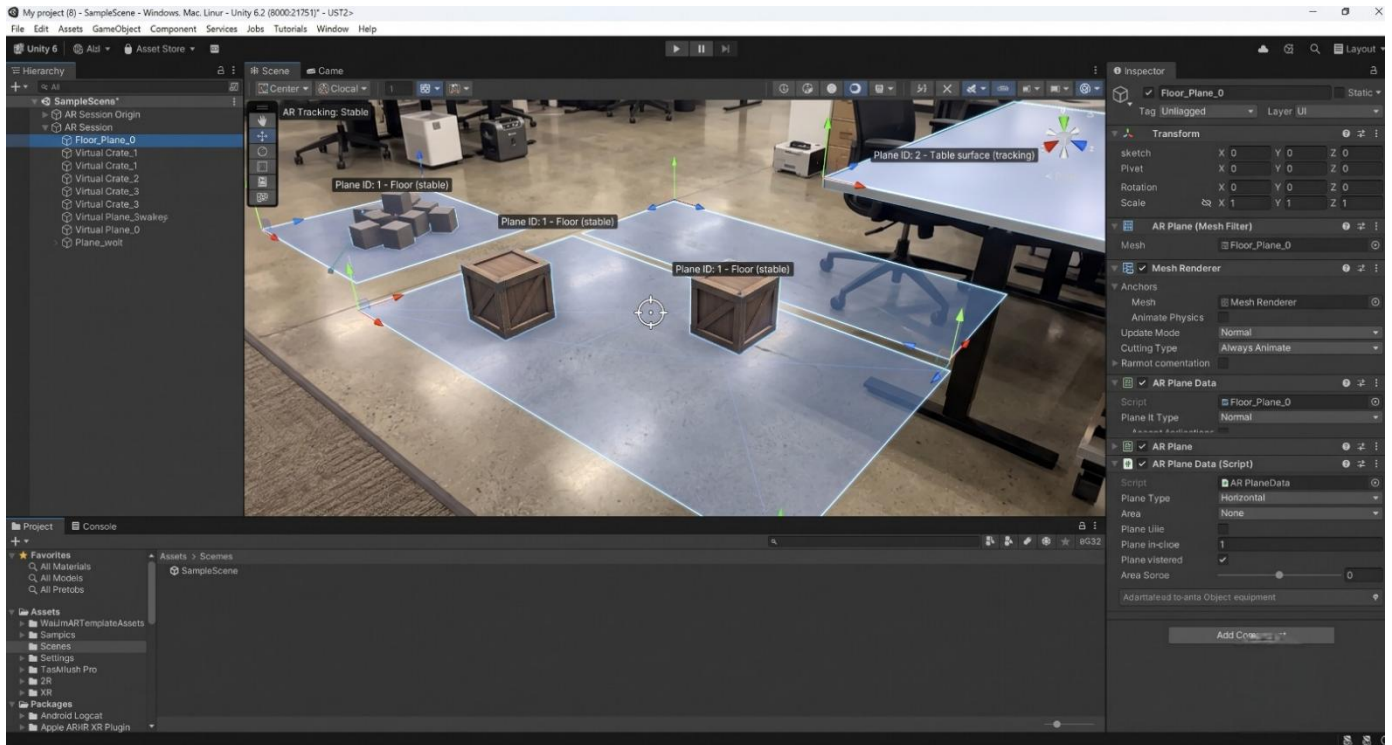
        if (touch.phase != TouchPhase.Began)
            return;

        if (raycastManager.Raycast(
            touch.position,
            hits,
            TrackableType.PlaneWithinPolygon))
        {
            Pose hitPose = hits[0].pose;

            Instantiate(
                objectToPlace,
                hitPose.position,
```

```
        hitPose.rotation
    );
}
}
```

OUTPUT:



RESULT:

Thus, the AR Camera was successfully configured, real-world planes were detected, and virtual objects were successfully placed on the detected planes using AR Foundation.

EX.NO: 05	IMAGE TRACKING DEMO, AR UI INTEGRATION	PAGE NO:
DATE:		

AIM:

To create an Image Tracking AR demo in Unity and integrate AR UI elements for displaying information and interacting with tracked images.

PROCEDURE:

1. Start Unity Hub and open the required AR project.
2. Open the Package Manager and verify that AR Foundation and the required XR plug-in are installed.
3. Create or open an AR scene and add an AR Session.
4. Add an XR Origin (AR) with an AR Camera to the scene.
5. Create a Reference Image Library for storing the images to be tracked.
6. Add the required target image to the Reference Image Library and configure its physical size.
7. Add an AR Tracked Image Manager to the XR Origin.
8. Assign the Reference Image Library to the Tracked Image Manager.
9. Create a 3D object or information panel that will appear when the target image is detected.
10. Configure the tracked image event to display the virtual object when the image is recognized.
11. Create a Canvas and add AR UI elements such as Text, Button, and Panel.
12. Position and customize the AR UI to display information related to the tracked image.
13. Build and run the AR application on a supported Android device and point the camera toward the target image.
14. Verify the demo by checking that the virtual object and AR UI appear correctly when the image is detected.

PROGRAM:

```
using UnityEngine;
using UnityEngine.XR.ARFoundation;

public class SimpleImageTracking : MonoBehaviour
{
    public ARTrackedImageManager imageManager;

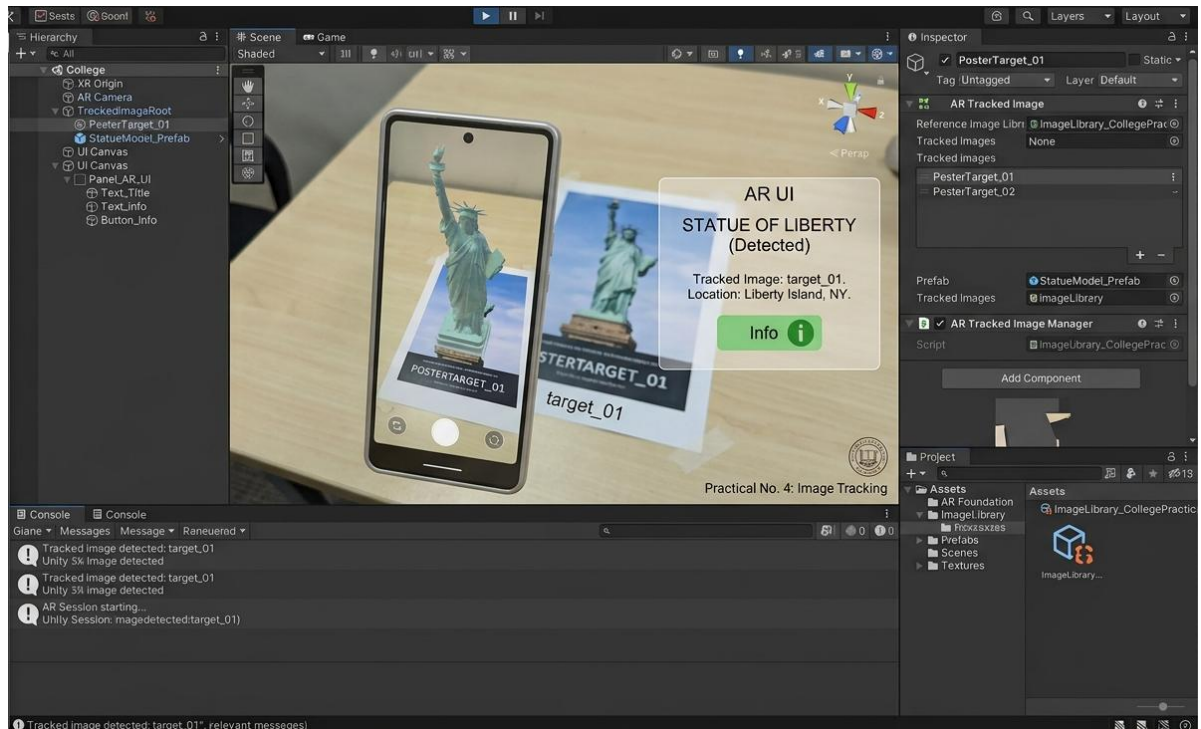
    private void OnEnable()
    {
        imageManager.trackedImagesChanged += OnImageChanged;
    }

    private void OnDisable()
    {
        imageManager.trackedImagesChanged -= OnImageChanged;
    }

    private void OnImageChanged(ARTrackedImagesChangedEventArgs args)
    {
        foreach (ARTrackedImage image in args.added)
        {
            Debug.Log("Image Detected: " + image.referenceImage.name);
        }

        foreach (ARTrackedImage image in args.updated)
        {
            Debug.Log("Image Updated: " + image.referenceImage.name);
        }
    }
}
```


OUTPUT:



RESULT:

Thus, an Image Tracking AR demo was successfully created, and AR UI elements were integrated to display information and provide interaction when the target image was detected.

AIM:

To set up a VR environment for Oculus/PC, add a VR Rig, and implement teleportation for movement within a virtual scene.

PROCEDURE:

1. Start Unity Hub and open the required VR project.
2. Open the Package Manager and install the required XR Plug-in Management and XR Interaction Toolkit packages.
3. Open Project Settings and enable the appropriate XR Plug-in provider for the target VR platform.
4. Configure the PC/Oculus platform according to the connected VR headset.
5. Create or open a 3D scene for the VR environment.
6. Add an XR Origin (VR) to the scene to provide the VR camera and controller setup.
7. Configure the Main Camera and VR controllers within the XR Origin.
8. Create a virtual environment by adding objects such as floors, walls, platforms, and other 3D objects.
9. Add a Teleportation Area to the floor or other suitable surfaces.
10. Configure the teleportation provider and controller input for teleport movement.
11. Set the required teleportation layers and interaction settings for the VR environment.
12. Connect the Oculus/VR headset to the PC and verify that the headset and controllers are detected.
13. Run the scene in Play mode and test VR interaction and teleportation using the controllers.
14. Verify the VR experience by moving around the virtual environment and confirming that teleportation works correctly.

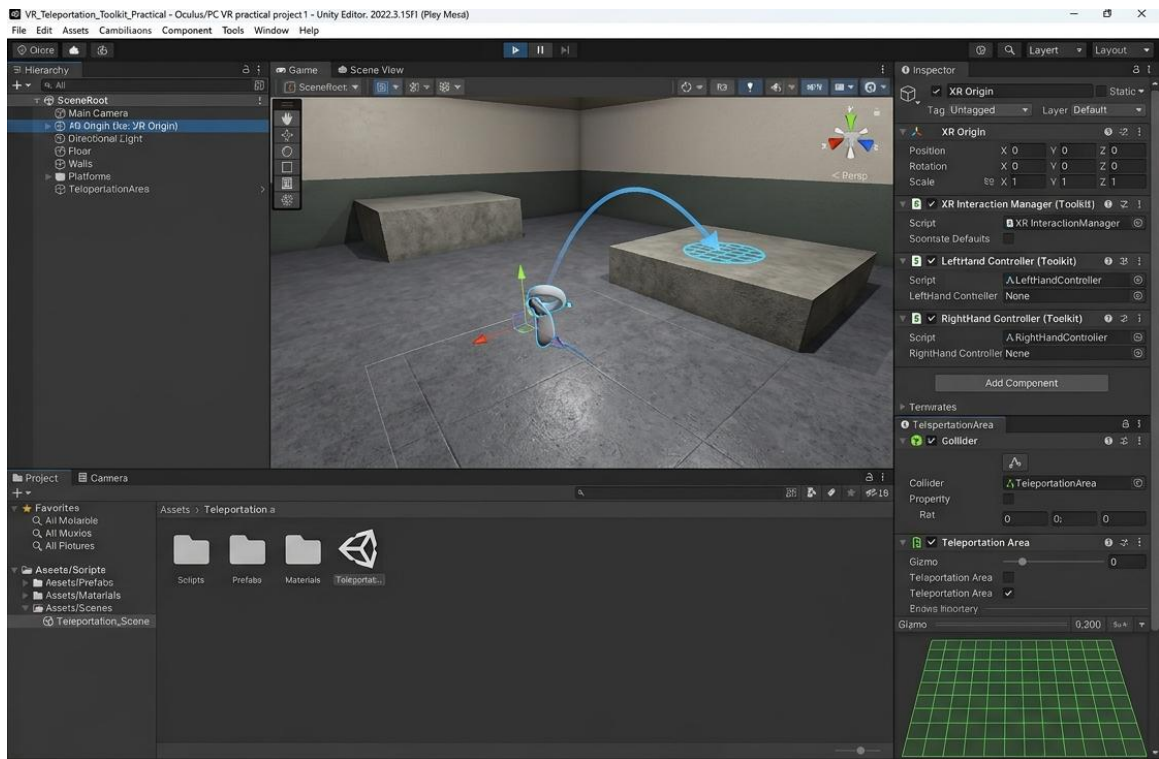
PROGRAM:

```
using UnityEngine;

public class TeleportPoint : MonoBehaviour
{
    public Transform player;

    public void TeleportPlayer()
    {
        player.position = transform.position;
    }
}
```

OUTPUT:



RESULT:

Thus, the VR environment for Oculus/PC was successfully configured, a VR Rig (XR Origin) was added, and teleportation-based movement was successfully implemented and tested.

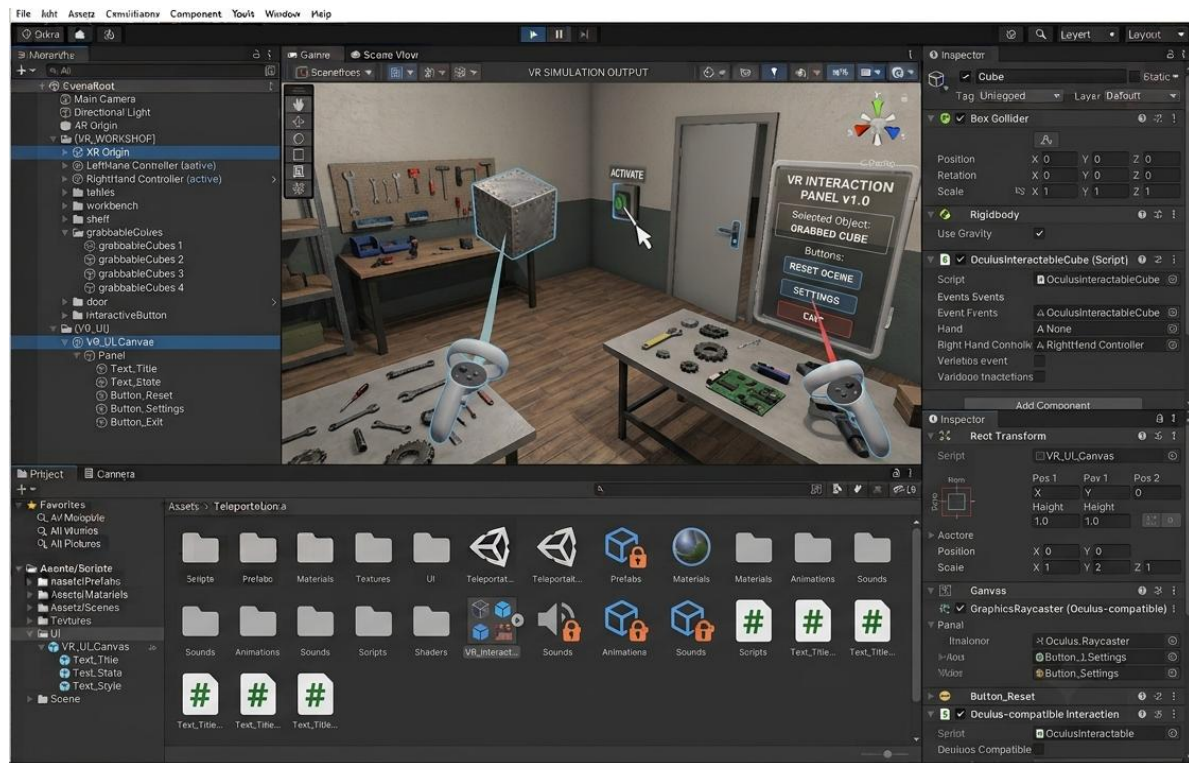
AIM:

To build a VR environment in Unity by adding interactive objects and user interface (UI) elements for an immersive and interactive VR experience.

PROCEDURE:

1. Start Unity Hub and open the existing VR project.
2. Open the VR scene containing the XR Origin and configured VR camera.
3. Create the VR environment by adding floors, walls, platforms, and other 3D objects.
4. Arrange the Game Objects in the Scene View to form a suitable virtual environment.
5. Add interactive objects such as buttons, cubes, doors, or other 3D models.
6. Add Collider components to the interactive objects to enable interaction.
7. Add XR Grab Interactable components to objects that need to be picked up or moved.
8. Configure the VR controllers with the required interaction and input settings.
9. Create a Canvas for displaying the VR user interface.
10. Add UI elements such as Text, Buttons, Panels, and Images to the Canvas.
11. Configure the UI elements and position them appropriately within the VR environment.
12. Set up interactions so that users can select or activate objects and UI buttons using VR controllers.
13. Run the VR scene and test object interaction and UI functionality using the headset and controllers.
14. Save and build the project, then verify that the VR environment, interactive objects, and UI work correctly.

OUTPUT:



RESULT:

Thus, a VR environment was successfully created with interactive objects and UI elements, allowing users to interact with virtual objects and interface components using VR controllers.

EX.NO: 08

DATE:

**ADD SPATIAL SOUND, OPTIMIZE MOBILE
PERFORMANCE**

PAGE NO:

AIM:

To add spatial sound to a Unity application and optimize the project for better performance on mobile devices.

PROCEDURE:

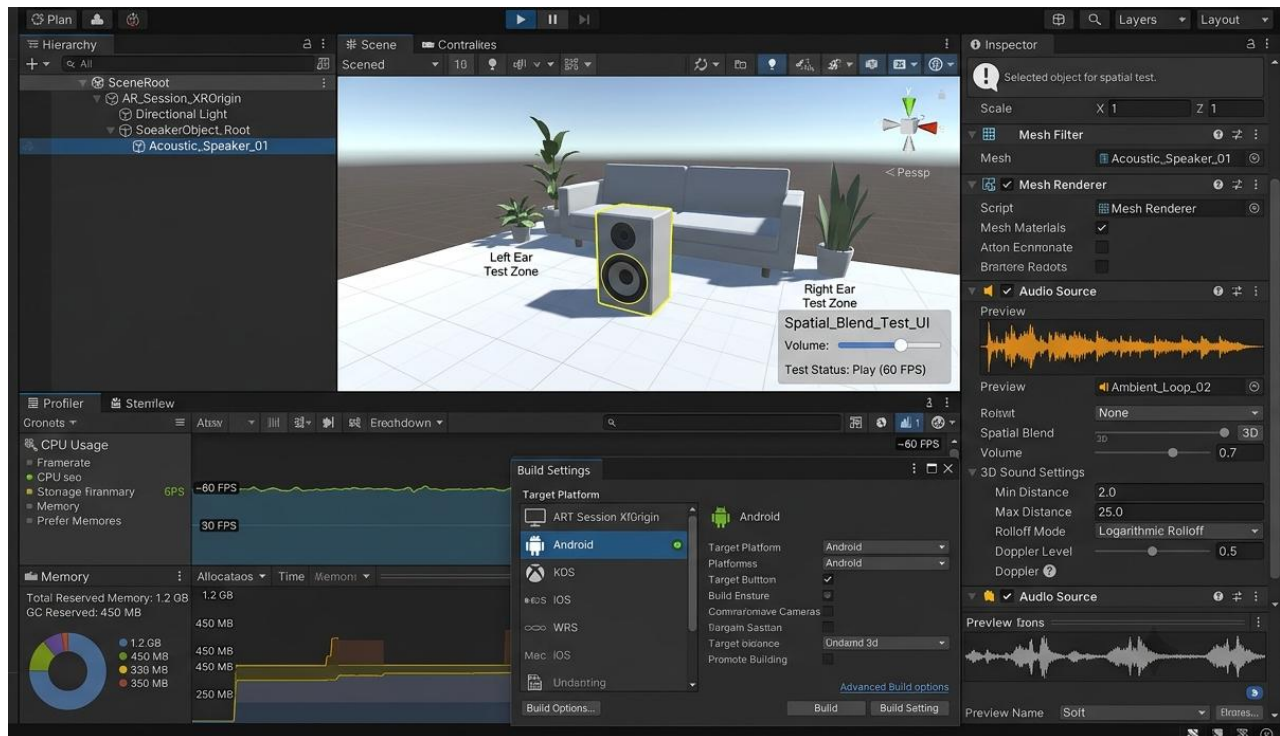
1. Start Unity Hub and open the required project.
2. Open the existing AR/VR scene and verify that the required camera and player setup are available.
3. Add an Audio Source component to the Game Object that will produce sound.
4. Assign an audio clip to the Audio Source and configure its playback settings.
5. Enable spatial sound and set the Audio Source to 3D mode.
6. Configure the spatial sound settings such as Min Distance, Max Distance, and Volume Rolloff.
7. Position the sound-producing object within the scene and test how the sound changes with distance.
8. Add suitable audio clips to important interactive objects to improve the immersive experience.
9. Open the mobile platform settings and configure the project for the target Android device.
10. Optimize 3D models and textures by reducing unnecessary polygons and texture sizes.
11. Reduce unnecessary Game Objects, components, and visual effects to lower processing requirements.
12. Optimize lighting and rendering settings to reduce GPU and CPU workload.
13. Test the application on a mobile device and monitor performance, frame rate, memory usage, and responsiveness.
14. Build the optimized application and verify that spatial sound works correctly while maintaining smooth mobile performance.

PROGRAM:

spatial sound using UnityEngine;

```
public class SpatialSound : MonoBehaviour
{
    void Start()
    {
        GetComponent().spatialBlend = 1f;
    }
}
```


OUTPUT:



RESULT:

Thus, spatial sound was successfully added to the Unity application, and the project was optimized for mobile performance to achieve smoother rendering, reduced resource usage, and improved responsiveness.

EX.NO: 09 DATE:	UI MENU IN AR/VR, INPUT MODULES, AR DEMO USING ALTERNATE WORKFLOWS BUILD AR DEMO (POSTER, CARD, MUSEUM, ETC...), CREATE VR SCENE (GALLERY, ESCAPE ROOM, ETC...)	PAGE NO:
--------------------------------------	--	-----------------

AIM:

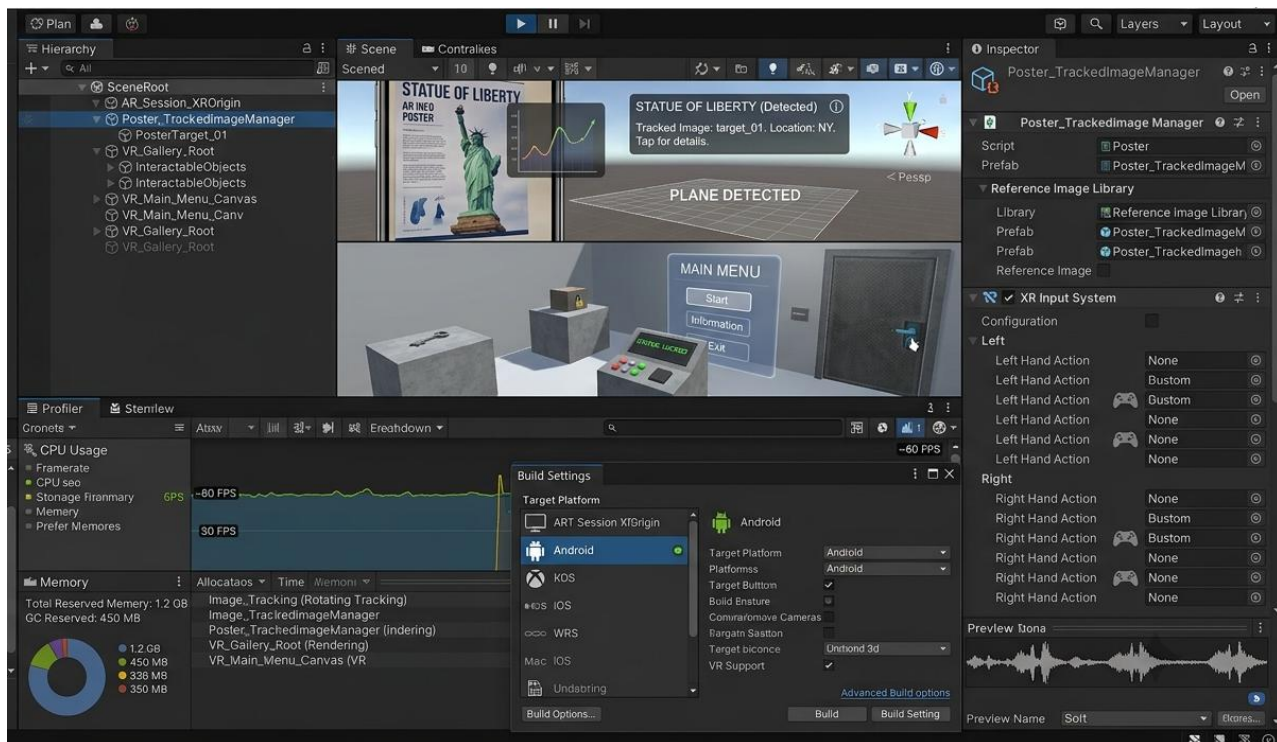
To create a UI menu in AR/VR, configure input modules, and develop AR and VR demos using alternate workflows such as posters, cards, museums, galleries, or escape rooms.

PROCEDURE:

1. Start Unity Hub and open the existing AR/VR project.
2. Create or open an AR/VR scene and verify that the required XR components are configured.
3. Create a Canvas and design a UI menu containing elements such as Buttons, Text, Panels, and Images.
4. Configure the UI Canvas according to the selected AR or VR environment.
5. Add and configure the appropriate Input Module to receive input from touch, mouse, keyboard, or VR controllers.
6. Connect UI buttons to required actions such as opening panels, displaying information, or changing scenes.
7. Create an AR demo using an alternate workflow, such as an image-tracked poster, card, or museum information board.
8. Add a Reference Image Library and include the selected poster or card as the tracking image.
9. Configure the AR Tracked Image Manager and associate virtual objects or information with the tracked image.

10. Test the AR demo by pointing the device camera toward the poster, card, or museum image and verifying the virtual content.
11. Create a separate VR scene such as a gallery or escape room using suitable 3D models, rooms, and interactive objects.
12. Add interactive elements such as doors, buttons, objects, displays, or clues to the VR scene.
13. Configure VR input and interactions so the user can select, grab, inspect, or activate objects using the VR controllers.
14. Run and test both AR and VR demos, verify the UI menus and input modules, and build the application for the target platform.

OUTPUT:



RESULT:

Thus, a UI menu and input modules were successfully integrated into the AR/VR application. An AR demo using a poster, card, or museum workflow and a VR scene such as a gallery or escape room were created and tested successfully.

EX.NO: 10 DATE:	PRESENTATION, DEMO DAY, FINAL TOUCHES	PAGE NO:
----------------------------------	--	-----------------

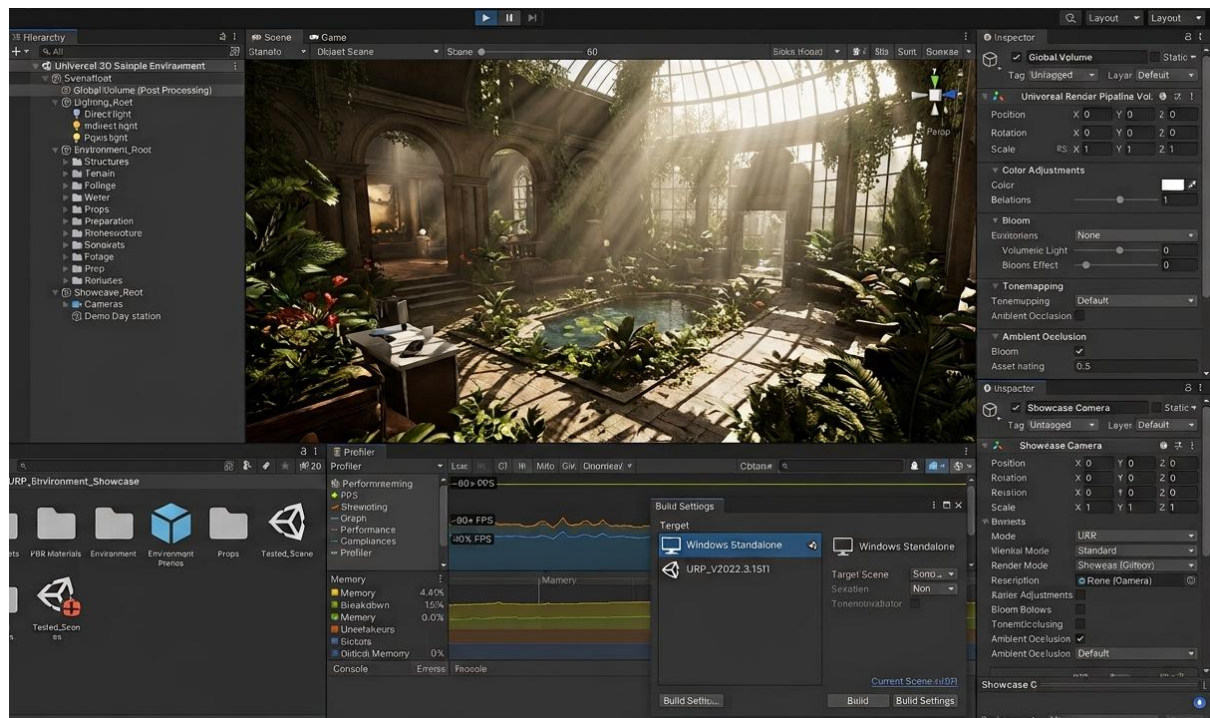
AIM:

To prepare the AR/VR project for presentation and demo day by applying final improvements, testing all features, and ensuring the application is ready for demonstration.

PROCEDURE:

1. Open the completed AR/VR project in Unity.
2. Review all project features and verify that the required modules are implemented.
3. Check the AR/VR scenes for missing Game Objects, components, assets, or references.
4. Test the UI menus and input modules to ensure that all controls respond correctly.
5. Test AR features such as image tracking, plane detection, and object placement.
6. Test VR features such as the VR Rig, teleportation, and interactive objects.
7. Verify spatial audio and ensure that sounds are triggered correctly.
8. Optimize the project by removing unnecessary assets and reducing performance overhead.
9. Build the final application for the target platform and test it on the required device.
10. Fix errors and bugs identified during the final testing process.
11. Prepare the presentation slides covering the project objective, methodology, features, implementation, and results.
12. Prepare the live demonstration and arrange the required software, devices, and project files.
13. Perform final touches such as improving UI appearance, scene arrangement, audio, and overall presentation quality.
14. Conduct a complete demo rehearsal and verify that the final project is ready for presentation and demo day.

OUTPUT:



RESULT:

Thus, the AR/VR project was successfully finalized and tested, with the presentation, live demonstration, and final project setup prepared for Demo Day.